Table 1: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means the denominator is faster; **bold** marks >1.1×.

Program	**ADT**	**SoA**	**End-to-end**					**Fold passes**			**Map passes**		
	fields	bufs	Am (s)	Ai (s)	Sm (s)	Am/Sm	Ai/Sm	Am (s)	Sm (s)	Am/Sm	Am (s)	Sm (s)	Am/Sm
OctTree	16	9	0.6741	–	0.6815	0.99×	–	0.3736	0.3287	**1.14**×	0.3005	0.3528	0.85×
Compiler	9	8	0.2940	1.101	0.2274	**1.29**×	**4.84**×	0.1347	0.0539	**2.50**×	0.1593	0.1735	0.92×
DBQuery	15	13	0.2650	0.3269	0.2535	1.05×	**1.29**×	0.1273	0.0880	**1.45**×	0.1378	0.1655	0.83×
DecisionTree	7	6	3.847	5.621	2.937	**1.31**×	**1.91**×	3.847	2.937	**1.31**×	–	–	–
DomTree	15	14	0.2058	0.3400	0.1480	**1.39**×	**2.30**×	0.1471	0.0644	**2.28**×	0.0587	0.0835	0.70×
KDTree	17	16	0.1788	0.2620	0.1385	**1.29**×	**1.89**×	0.1788	0.1385	**1.29**×	–	–	–
LinearListReduction	11	11	0.0298	0.1391	0.0051	**5.80**×	**27.04**×	0.0298	0.0051	**5.80**×	–	–	–
List	2	2	0.5785	–	0.6434	0.90×	–	0.3252	0.3015	1.08×	0.2533	0.3418	0.74×
MonoTree	3	2	0.0746	0.1420	0.0663	**1.13**×	**2.14**×	0.0177	0.0225	0.79×	0.0569	0.0438	**1.30**×
ObjectGraph	5	4	0.2349	0.6054	0.2439	0.96×	**2.48**×	0.0907	0.0807	**1.12**×	0.1442	0.1632	0.88×
PiecewiseFunctions	8	7	0.4966	0.5625	0.3552	**1.40**×	**1.58**×	0.1379	0.0911	**1.51**×	0.3587	0.2642	**1.36**×
TernaryTree	5	3	0.1146	0.1444	0.1102	1.04×	**1.31**×	0.0302	0.0291	1.04×	0.0844	0.0811	1.04×
Trie	10	9	0.2169	0.2973	0.2032	1.07×	**1.46**×	0.0672	0.0325	**2.07**×	0.1496	0.1707	0.88×
ColorOctree	25	18	0.0909	0.1201	0.0773	**1.18**×	**1.55**×	0.0909	0.0773	**1.18**×	–	–	–

Table 2: Baseline Gibbon comparison (times only). Shown variants: AoS-mut, AoS-imm, SoA-mut. Times are full executable end-to-end wall time per run (s), not pass-sum time. **Bold** marks the fastest time across shown variants.

Program	**Am**	**Ai**	**Sm**
Compiler	0.5404	1.752	**0.4749**
DBQuery	0.6228	0.6216	**0.5541**
DecisionTree	4.272	6.090	**3.397**
DomTree	**0.6000**	0.7602	0.6038
KDTree	**0.4308**	0.5384	0.4469
LinearListReduction	**0.2139**	0.8089	0.3597
List	**0.9602**	*OOM*	1.150
MonoTree	**0.2365**	0.3284	0.2607
ObjectGraph	**0.4742**	0.9204	0.5084
OctTree	**3.242**	3.786	7.773
PiecewiseFunctions	0.8049	0.9429	**0.6742**
TernaryTree	**0.3269**	0.3595	0.3497
Trie	**0.3560**	0.5844	0.4733
ColorOctree	**0.4763**	0.5222	0.8028

Table 3: Baseline Gibbon comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut. Speedups are computed from full executable end-to-end wall time per run. >1× means the denominator is faster.

Program	**Am/Sm**	**Ai/Am**	**Ai/Sm**
Compiler	**1.14**×	**3.24**×	**3.69**×
DBQuery	**1.12**×	1.00×	**1.12**×
DecisionTree	**1.26**×	**1.43**×	**1.79**×
DomTree	0.99×	**1.27**×	**1.26**×
KDTree	0.96×	**1.25**×	**1.20**×
LinearListReduction	0.59×	**3.78**×	**2.25**×
List	0.83×	–	–
MonoTree	0.91×	**1.39**×	**1.26**×
ObjectGraph	0.93×	**1.94**×	**1.81**×
OctTree	0.42×	**1.17**×	0.49×
PiecewiseFunctions	**1.19**×	**1.17**×	**1.40**×
TernaryTree	0.93×	**1.10**×	1.03×
Trie	0.75×	**1.64**×	**1.23**×
ColorOctree	0.59×	1.10×	0.65×

Table 4: Per-pass performance for `OctTree`, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
barnesHutPotential	F	11/16	31%	0.0413	0.0580	**0.0392**	1.05×	**1.40×**	**1.48×**
countActive	F	10/16	38%	0.0372	0.0451	**0.0290**	**1.28×**	**1.21×**	**1.55×**
countParticles	F	8/16	50%	0.0359	0.0436	**0.0270**	**1.33×**	**1.22×**	**1.61×**
fmmPotential	F	12/16	25%	0.0977	0.0964	**0.0964**	1.01×	0.99×	1.00×
sumEnergy	F	12/16	25%	0.0361	0.0543	**0.0328**	**1.10×**	**1.50×**	**1.66×**
sumMass	F	10/16	38%	0.0345	0.0430	**0.0270**	**1.28×**	**1.25×**	**1.59×**
clearFlags	M	15/16	6%	0.1539	**0.1485**	0.1761	0.87×	0.96×	0.84×
scaleEnergy	M	16/16	0%	**0.1466**	0.1474	0.1767	0.83×	1.01×	0.83×
paletteEntriesQuantized	F	13/25	48%	0.0439	0.0545	**0.0340**	**1.29×**	**1.24×**	**1.60×**
quantizationErrorProxy	F	10/25	60%	0.0470	0.0656	**0.0433**	1.09×	**1.39×**	**1.51×**
Total				**0.6741**	0.7564	0.6815	0.99×	**1.12×**	**1.11×**
Geomean						**1.10×**	**1.20×**	**1.33×**	

Table 5: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
blockCountPass	F	2/9	78%	0.0118	0.0778	**0.0020**	**6.00×**	**6.59×**	**39.52×**
branchStatsPass	F	2/9	78%	0.0115	0.0784	**0.0034**	**3.36×**	**6.80×**	**22.83×**
castInstCountPass	F	3/9	67%	0.0178	0.0783	**0.0014**	**13.14×**	**4.41×**	**57.95×**
has cycle	F	4/9	56%	0.0344	0.0921	**0.0318**	1.08×	**2.68×**	**2.89×**
instCountPass	F	2/9	78%	0.0119	0.0778	**0.0020**	**5.88×**	**6.55×**	**38.47×**
latencyModelPass	F	3/9	67%	0.0117	0.0792	**0.0030**	**3.89×**	**6.75×**	**26.26×**
memoryOpStatsPass	F	3/9	67%	0.0115	0.0785	**0.0038**	**3.01×**	**6.81×**	**20.51×**
throughputModelPass	F	3/9	67%	0.0117	0.0787	**0.0029**	**3.99×**	**6.71×**	**26.79×**
verifyIR	F	9/9	0%	0.0124	0.0673	**0.0035**	**3.51×**	**5.45×**	**19.15×**
stripSideEffectsPass	M	7/9	22%	**0.0799**	0.1857	0.0865	0.92×	**2.32×**	**2.15×**
targetReturnPass	M	9/9	0%	**0.0794**	0.2074	0.0870	0.91×	**2.61×**	**2.38×**
Total				0.2940	1.101	**0.2274**	**1.29×**	**3.75×**	**4.84×**
Geomean						**3.08×**	**4.85×**	**14.93×**	

Table 6: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
countJoins	F	3/15	80%	0.0199	0.0290	**0.0111**	**1.80×**	**1.46×**	**2.62×**
filterSelectivitySkew	F	5/15	67%	0.0197	0.0282	**0.0124**	**1.58×**	**1.43×**	**2.27×**
hashJoinPressure	F	5/15	67%	0.0213	0.0327	**0.0131**	**1.63×**	**1.54×**	**2.50×**
sumCost	F	6/15	60%	0.0198	0.0286	**0.0147**	**1.35×**	**1.45×**	**1.95×**
sumMemory	F	6/15	60%	0.0198	0.0281	**0.0147**	**1.35×**	**1.42×**	**1.91×**
sumRows	F	6/15	60%	0.0268	0.0304	**0.0221**	**1.22×**	**1.13×**	**1.38×**
clearQueryFlags	M	14/15	7%	**0.0688**	0.0749	0.0818	0.84×	1.09×	0.92×
scaleCosts	M	15/15	0%	**0.0689**	0.0750	0.0837	0.82×	1.09×	0.90×
Total				0.2650	0.3269	**0.2535**	1.05×	**1.23×**	**1.29×**
Geomean						**1.28×**	**1.31×**	**1.68×**	

Table 7: Per-pass performance for `DecisionTree`, ADT has 7 fields, SoA uses 6 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
classify Batch	F	5/7	29%	3.312	4.253	**2.592**	**1.28×**	**1.28×**	**1.64×**
classify Depth	F	4/7	43%	0.0462	0.0605	**0.0370**	**1.25×**	**1.31×**	**1.64×**
classify tree	F	5/7	29%	0.0464	0.0603	**0.0381**	**1.22×**	**1.30×**	**1.58×**
countFeatureUses	F	3/7	57%	0.0495	0.1420	**0.0317**	**1.56×**	**2.87×**	**4.48×**
countLeaves	F	2/7	71%	0.0469	0.1402	**0.0248**	**1.89×**	**2.99×**	**5.65×**
countNodes	F	2/7	71%	0.0472	0.1400	**0.0265**	**1.78×**	**2.97×**	**5.29×**
countSmallLeaves	F	3/7	57%	0.0504	0.1357	**0.0323**	**1.56×**	**2.69×**	**4.20×**
inferenceCost	F	2/7	71%	0.0498	0.1387	**0.0290**	**1.72×**	**2.79×**	**4.79×**
sumImpurity	F	3/7	57%	0.0495	0.1394	**0.0321**	**1.54×**	**2.81×**	**4.34×**
sumPathLengths	F	3/7	57%	0.0517	0.1383	**0.0336**	**1.54×**	**2.68×**	**4.11×**
sumSamples	F	3/7	57%	0.0475	0.1344	**0.0317**	**1.50×**	**2.83×**	**4.24×**
treeDepth	F	2/7	71%	0.0504	0.1386	**0.0282**	**1.78×**	**2.75×**	**4.91×**
Total				3.847	5.621	**2.937**	**1.31×**	**1.46×**	**1.91×**
Geomean						**1.54×**	**2.32×**	**3.57×**	

Table 8: Per-pass performance for `DomTree`, ADT has 15 fields, SoA uses 14 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
count styled	F	3/15	80%	0.0364	0.0699	**0.0116**	**3.15×**	**1.92×**	**6.05×**
find max Bottom	F	5/15	67%	0.0367	0.0694	**0.0229**	**1.60×**	**1.89×**	**3.02×**
SumArea	F	6/15	60%	0.0377	0.0699	**0.0182**	**2.07×**	**1.86×**	**3.84×**
sumTextWidth	F	3/15	80%	0.0363	0.0702	**0.0117**	**3.09×**	**1.94×**	**5.99×**
computeWidths	M	14/15	7%	**0.0297**	0.0302	0.0466	0.64×	1.02×	0.65×
scaleLayout	M	15/15	0%	**0.0290**	0.0304	0.0370	0.78×	1.05×	0.82×
Total				0.2058	0.3400	**0.1480**	**1.39×**	**1.65×**	**2.30×**
Geomean						**1.59×**	**1.55×**	**2.47×**	

Table 9: Per-pass performance for `KDTree`, ADT has 17 fields, SoA uses 16 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
countInRange tight_box	F	13/17	24%	0.0278	0.0430	**0.0198**	**1.40×**	**1.55×**	**2.17×**
Find nearest Neighbour	F	13/17	24%	0.0277	0.0421	**0.0212**	**1.31×**	**1.52×**	**1.98×**
photonMappingBenchmark	F	12/17	29%	0.0427	0.0508	**0.0404**	1.06×	**1.19×**	**1.26×**
pointCloudNeighborhood	F	11/17	35%	0.0248	0.0414	**0.0164**	**1.51×**	**1.67×**	**2.52×**
sumMassInRange	F	14/17	18%	0.0272	0.0429	**0.0199**	**1.37×**	**1.58×**	**2.16×**
twoPointCorrelation bin_8_16	F	11/17	35%	0.0287	0.0419	**0.0208**	**1.38×**	**1.46×**	**2.02×**
Total				0.1788	0.2620	**0.1385**	**1.29×**	**1.47×**	**1.89×**
Geomean						**1.33×**	**1.49×**	**1.97×**	

Table 10: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
reduction	F	2/11	82%	0.0298	0.1391	**0.0051**	**5.80×**	**4.66×**	**27.04×**
Total				0.0298	0.1391	**0.0051**	**5.80×**	**4.66×**	**27.04×**
Geomean						**5.80×**	**4.66×**	**27.04×**	

Table 11: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
length List	F	1/2	50%	0.0481	*OOM*	**0.0261**	**1.84×**	–	–
sumList	F	2/2	0%	0.1385	*OOM*	**0.1380**	1.00×	–	–
sumList tail recursive	F	2/2	0%	0.1386	*OOM*	**0.1374**	1.01×	–	–
add1 List	M	2/2	0%	**0.2533**	*OOM*	0.3418	0.74×	–	–
Total				**0.5785**	–	0.6434	0.90×	–	–
Geomean						1.08×	–	–	

Table 12: Per-pass performance for `MonoTree`, ADT has 3 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
sumTree	F	3/3	0%	**0.0091**	0.0402	0.0113	0.81×	**4.40×**	**3.56×**
sumTree TailRec	F	3/3	0%	**0.0086**	0.0403	0.0112	0.76×	**4.70×**	**3.59×**
add1Tree	M	3/3	0%	0.0569	0.0615	**0.0438**	**1.30×**	1.08×	**1.40×**
Total				0.0746	0.1420	**0.0663**	**1.13×**	**1.90×**	**2.14×**
Geomean						0.93×	**2.82×**	**2.62×**	

Table 13: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
countLargeItems	F	3/5	40%	0.0126	0.0446	**0.0104**	**1.21×**	**3.55×**	**4.31×**
countMarked	F	3/5	40%	0.0131	0.0447	**0.0108**	**1.21×**	**3.42×**	**4.13×**
countSurvivors	F	4/5	20%	0.0135	0.0446	**0.0131**	1.03×	**3.30×**	**3.40×**
deadBytes	F	4/5	20%	0.0133	0.0440	**0.0126**	1.05×	**3.31×**	**3.48×**
liveBytes	F	4/5	20%	0.0132	0.0431	**0.0127**	1.04×	**3.27×**	**3.40×**
sumObjIds	F	3/5	40%	0.0122	0.0433	**0.0102**	**1.20×**	**3.56×**	**4.26×**
totalHeapSize	F	3/5	40%	0.0129	0.0435	**0.0109**	**1.18×**	**3.37×**	**3.98×**
sweepUnmarked	M	5/5	0%	**0.0714**	0.2086	0.0807	0.88×	**2.92×**	**2.58×**
touchHotObjects	M	5/5	0%	**0.0728**	0.0889	0.0825	0.88×	**1.22×**	1.08×
Total				**0.2349**	0.6054	0.2439	0.96×	**2.58×**	**2.48×**
Geomean						1.07×	**2.98×**	**3.19×**	

Table 14: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
autorefineMaxLevel	F	4/8	50%	0.0211	0.0462	**0.0193**	1.09×	**2.19×**	**2.39×**
compressMass	F	3/8	62%	0.0193	0.0458	**0.0116**	**1.67×**	**2.38×**	**3.96×**
lbDeuxLoadProxy	F	5/8	38%	0.0211	0.0475	**0.0181**	**1.16×**	**2.25×**	**2.62×**
norm2Estimate	F	5/8	38%	0.0248	0.0556	**0.0179**	**1.38×**	**2.24×**	**3.11×**
pmapCutHistogram	F	4/8	50%	0.0320	0.0564	**0.0127**	**2.52×**	**1.76×**	**4.43×**
truncateTolViolations	F	3/8	62%	0.0196	0.0469	**0.0114**	**1.71×**	**2.39×**	**4.10×**
addConstPW	M	8/8	0%	**0.1186**	0.1317	0.1337	0.89×	**1.11×**	0.99×
diffPW	M	8/8	0%	0.2401	0.1324	**0.1304**	**1.84×**	0.55×	1.02×
Total				0.4966	0.5625	**0.3552**	**1.40×**	**1.13×**	**1.58×**
Geomean						**1.46×**	**1.69×**	**2.47×**	

Table 15: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
sum tree	F	5/5	0%	0.0302	0.0507	**0.0291**	1.04×	**1.68×**	**1.75×**
add 1 tree	M	5/5	0%	0.0844	0.0937	**0.0811**	1.04×	**1.11×**	**1.15×**
Total				0.1146	0.1444	**0.1102**	1.04×	**1.26×**	**1.31×**
Geomean						1.04×	**1.37×**	**1.42×**	

Table 16: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Am/Sm**	**Ai/Am**	**Ai/Sm**
autocompleteTopKProxy	F	5/10	50%	0.0150	0.0278	**0.0084**	**1.78**×	**1.86**×	**3.31**×
countLazyNodes	F	4/10	60%	0.0134	0.0283	**0.0066**	**2.03**×	**2.11**×	**4.28**×
countTerminals	F	3/10	70%	0.0128	0.0263	**0.0058**	**2.21**×	**2.05**×	**4.52**×
sumPrefixFreq	F	3/10	70%	0.0131	0.0267	**0.0059**	**2.21**×	**2.03**×	**4.48**×
sumSubtreeHints	F	3/10	70%	0.0129	0.0266	**0.0057**	**2.26**×	**2.06**×	**4.66**×
decayTrieStats	M	10/10	0%	**0.0748**	0.0804	0.0868	0.86×	1.07×	0.93×
resetTraversalState	M	10/10	0%	**0.0748**	0.0812	0.0840	0.89×	1.09×	0.97×
Total				0.2169	0.2973	**0.2032**	1.07×	**1.37**×	**1.46**×
Geomean						**1.63**×	**1.69**×	**2.75**×	